

Dia 3 Python e AI

Módulo NumPy:

- Para usarmos as funções do módulo basta escrever: `import numpy`. Dessa forma, podemos manipular os arrays.

Termina no último.

Ex: `print(arr[1:3]); print(arr[2:1])`

- Também é possível definir os passos com o terceiro especificado.

Ex: `print(arr[0:3:2])` → anda de 2 em 2.

• `array()`: Gera um array com os elementos dos argumentos. Podemos escolher a quantidade de dimensões com `ndmin`.

Ex: `arr = np.array([1,2,3,4], ndmin=5)`

• `ndim`: Retorna o número de dimensões do objeto.

- Para acessar basta usar os colchetes com o índice da posição. Para mais dimensões, basta especificar dentro dos colchetes como em C. Com valores negativos acessamos os últimos elementos.

Ex: `arr = np.array([1,2,3,4], [5,6,7,8])`

`print(arr[1,-1])` → 8

`print(arr[0,0])`

`print(arr[0,0:3])` → [1,2,3]

Slicing: Pegar elementos de um índice até outro. `[X:X]`

Ex: `print(arr[0:3])`

- Caso o primeiro não seja especificado começa do início. Caso o segundo não seja especificado

• `dtype`: Retorna o tipo do array. Na função `array()` ele é um argumento opcional.

Ex: `arr = np.array([1,2,3,4], dtype="S")`

`print(arr.dtype)`

- Com `i, u, f, S` e `U` podemos definir o tamanho também (qtd de bytes).

- Se o tipo não corresponder com o que foi pedido após a correção do tipo retornará um erro.

Ex: `arr = np.array('a', 1, 2, dtype='i')`

• `astype()`: Transforma os dados no tipo escolhido se possível.

Ex: `n = arr.astype(bool)`

• `copy()`: Copia o array apenas.

• `view()`: Passa um "ponteiro" altera a original quando há alterações na view e ao contrário também.

base: Usamos para descobrir se um dado pertence ao objeto ou não. Se pertencer retorna `NONE`, caso contrário, retorna o original.

shape: Retorna uma tuple com cada index contendo a qtd de elementos da dimensão, da última para a primeira.

reshape(): Podemos redimensionar um array.

Ex: `arr.reshape(4,3)` ~> 

OBS: A conta de elementos deve bater com as posições nas dimensões ($4 \cdot 3 = 12$ elementos).

Retorna uma view. Podemos colocar -1 para completar as dimensões.

Ex: `arr = arr.reshape(2,2,-1)`

Também podemos retornar a 1 dimensão

Ex: `arr = arr.reshape(-1)`

Repetições: Podemos usar os laços normalmente, porém arrays de várias dimensões pode gerar poluição visual com vários for's aninhados.

Método `np.nditer()`: Percorre cada elemento de um array, independente da qtd de dimensões.

Ex: `for x in np.nditer(arr):
 print(x)`

Método `np.ndenumerate()`: Enumera as iterações. A função retorna um tuple com a enumeração e o elemento.

Ex: `for idx, x in np.ndenumerate(arr):`

Filtragem em arrays: Podemos filtrar valores em um array usando vetores auxiliares com valores booleanos ou usar chaves com uma condição.

Ex `arr`

`arr = np.array([1,2,3,4,5,6])`

`newarr = arr[arr > 3] ~> [4,5,6]`

`filter_arr = arr % 2 == 0`

`newarr = arr[filter_arr]`

Criação de arrays preenchidos:

`np.zeros()`: Retorna um array com zeros com tamanho e dimensões indicadas com uma tuple nos argumentos e o tipo também pode ser selecionado com dtype. (shape)

`np.ones()`: Funciona igual `np.zeros`, porém com 1's

`np.empty()`: Igual os outros, porém com lixo.

`np.arange()`: Cria números sequenciais em array.

`np.linspace()`: Cria um array com ^{uniforme} um espaço entre números em um intervalo predeterminado.

- Podemos usar as operações normais, como: +, -, *, / (Broadcasting). (Funciona entre arrays e números escalares)

`min()`: Menor número do array

`max()`: Maior número do array

`sum()`: Soma os elementos do array

`mean()`: Média dos elementos dada axis.

Números aleatórios em arrays:

`rand = np.random.default_rng()` # semente random

`rand.random()`: Float aleatório

`rand.integers()`: Int aleatório.